Abstract

The Video Graphics Array (VGA) standard, developed by IBM in 1987, serves as a widely adopted video adapter interface. VGA connections use a 15-pin DE15 adapter, with both male and female connectors. For this project, five key pins are important: three for the primary colors (red, green, and blue), and two for video synchronization (HSync and VSync).

Project 2 of the COE758 Digital Systems Engineering course involved creating a simple video game processor following the VGA standard and setting up input/output device interfacing on the Spartan-3E FPGA board. The goal of this project was to provide a foundational understanding of video output and VGA functionality, along with practical experience using both onboard circuits and external I/O devices, specifically the VGA monitor.

The project was developed in Xilinx ISE CAD software using VHDL programming. The VHDL code implemented essential components to manage VGA parameters and to display the correct images on a VGA monitor. A 25MHz clock signal, derived from the board's 50MHz oscillator. The derived clock signal was used to ensure a 60Hz refresh rate for the display.

The VHDL code was then used to program the Spartan-3E board, allowing it to display content on the VGA monitor and serve as the game controller through onboard switches. Ultimately, implementing this simple game provided valuable experience in adapting signals for VGA monitors and using FPGA boards for video output and VGA-related tasks.

Introduction

The simple video game implemented in Project Two was a simplified version of the classic "Pong" game developed by Atari. The game featured three dynamic elements. Two players could control paddles, a blue paddle on the left and a magenta paddle on the right. They could be controlled using the onboard switches, SW1 and SW3. Moving the switches upward moves the paddle up, while moving them downward shifts the paddle down. The third dynamic element was a yellow ball that traveled across the screen, changing its velocity upon interacting with different areas of the display. When the ball enters a goal on either the left or right side of the screen, it turns red and disappears, only to reappear at the center of the screen. The x and y pixel positions of the paddles and the ball were continuously tracked to determine interactions, such as collisions between the paddles, the white border, and the ball.

The vertical and horizontal pixel areas were tracked using counters for both dimensions, which reset based on the VGA parameters outlined in **Table 2** and **Table 3**. These pixel counts also determined the HSync and VSync signals required for synchronization. Additionally, the pixel count served as references for displaying the game's static elements, such as the green

background and white border. Dynamic elements, like the paddles and ball, were rendered using their coordinates in conjunction with horizontal and vertical pixel trackers. Colors were managed using three separate 8-bit arrays corresponding to the red, green, and blue signals. These digital signals were converted to analog by interfacing with the Spartan-3E board.

Horizontal and vertical pixel position was tracked based on the movement of pixel addressing. As shown in **Figure 1**, the display screen was treated as an M x N matrix, where a single frame was completed when all pixels in the matrix were addressed. The process starts at the top-left corner of the screen, moving horizontally across the row. Once the last pixel in the row was reached, the horizontal position was reset to the leftmost pixel, and the vertical position was incremented by one row. This cycle repeated until the bottom of the screen was reached. During the horizontal and vertical retrace periods, three key events occur before drawing resumes. First, the front porch cleared residual signals before the sync pulse. Next, the HSync or VSync signals synchronized the drawing position. Finally, the back porch provides time for the drawing mechanism to return to its starting position.

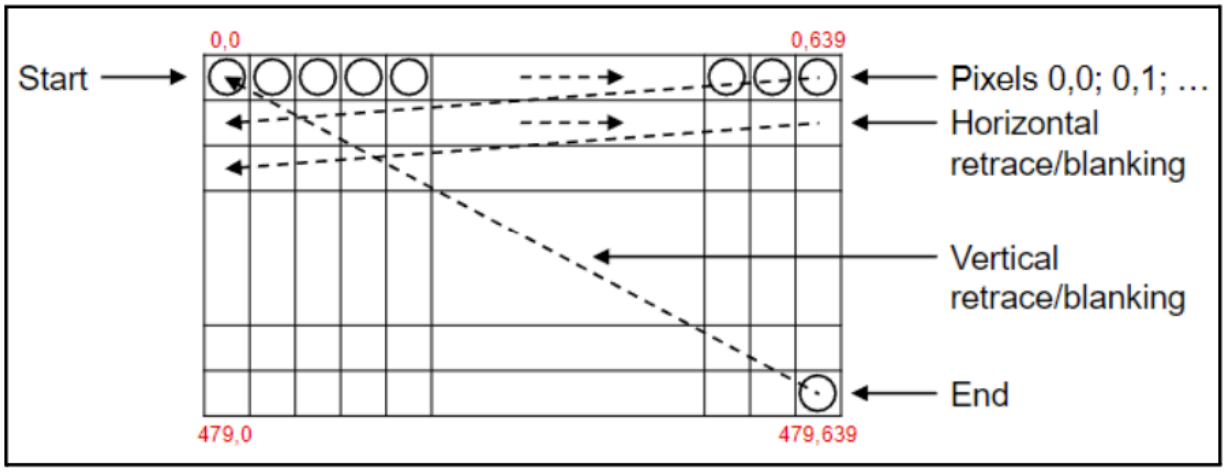

Figure 1: Diagram of pixel “movement” for one frame of an image

Port Mapping

The following tables display the signals generated by either the Spartan-3E board or the VHDL program in order for implementation of the simple video game and its display on a VGA monitor.

Table 1: Signals generated by the Spartan-3E board

Output signals by Spartan-3E	Pin location on Spartan-3E
50MHz clock signal (clk)	c9
Switch 0 (SW0)	L13
Switch 1 (SW1)	L14
Switch 2 (SW2)	H18
Switch 3 (SW3)	N17

Table 2: Signals generated by the VHDL program

Input signals by Spartan-3E	Pin location on Spartan-3E
HSync Signal (H)	c5
VSync Signal (V)	d5
25MHz clock signal (DAC_CLK)	a4
Red color channel (Rout[7 downto 0])	b16, a16, d14, c14, b14, a14, b13, a13
Green color channel (Gout[7 downto 0])	f9, e9, d11, c11, f11, e11, e12, f12
Blue color channel (Bout[7 downto 0])	a6, b6, e7, f7, d7, c7, f8, e8

Device Description / Design

Symbols

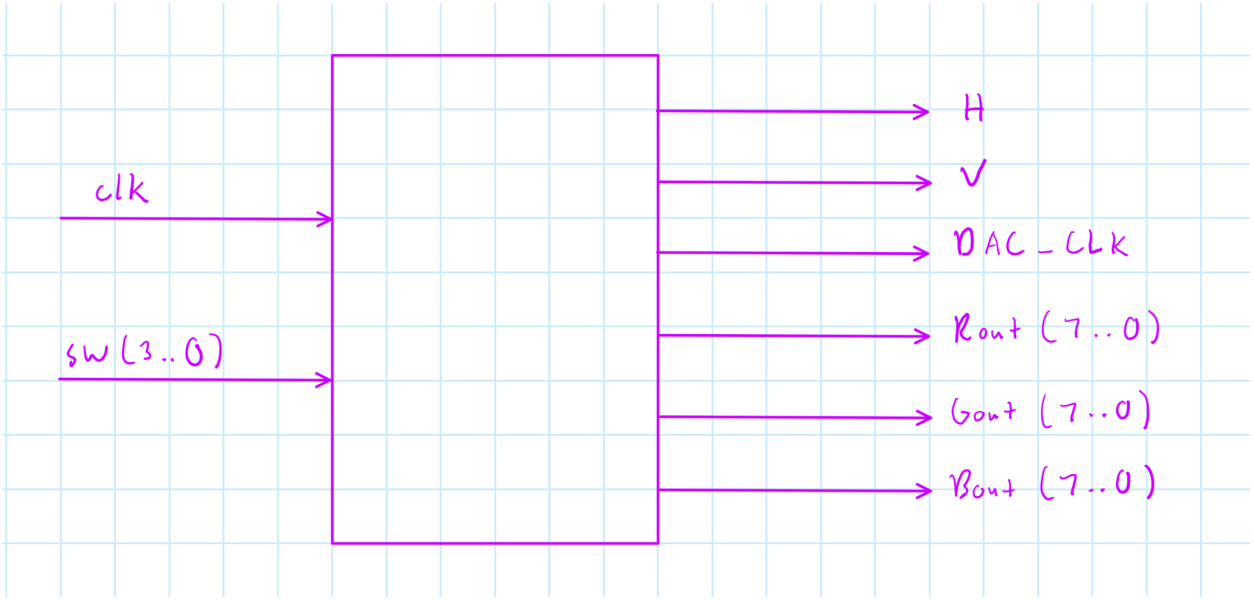

Figure 2: Symbol diagram of the Pong Game

The symbol diagram has two inputs, the clock (clk) and the switches (SW0, SW1, SW2, and SW3). It has 6 outputs, horizontal and vertical synchronization (H and V), the pixel clock (DAC_CLK), and colors (Rout, Gout, and Bout).The VHDL program was implemented entirely within a single top-level module, without relying on any externally defined components. This module handled all necessary processes, utilizing user input from the onboard switches and the 50MHz clock to generate synchronization signals, the DAC_CLK clock signal at the required frequency, and the red, green, and blue digital signals.

VGA Specification

Table 3: VGA Horizontal Parameters

Parameter	**Clock Cycles**
Complete Line	800
Front Porch	16
Sync Pulse	96
Back Porch	48
Display Image Area	640

Table 4: VGA Vertical Parameters

Parameter	**Lines**
Complete Line	525
Front Porch	10
Sync Pulse	2
Back Porch	33
Display Image Area	480

Block Diagrams

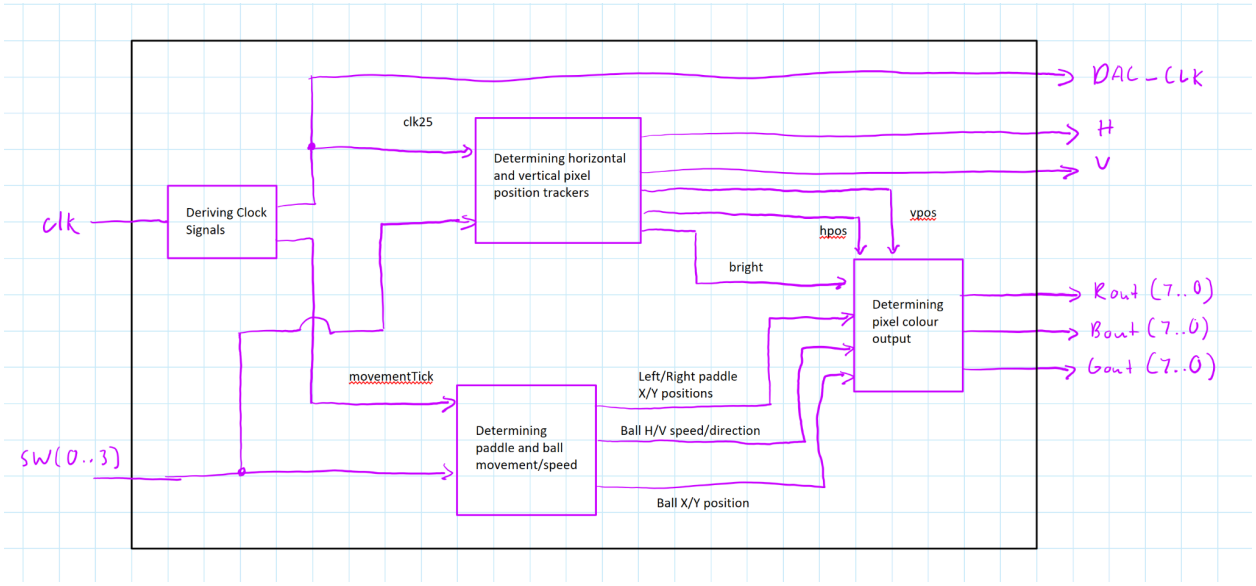

Figure 3: Block diagram of the Pong Game

The program begins by generating the necessary clock signals: a 25MHz clock (clk25) and a slower clock signal (movementTick) used to update the positions of the paddles and the ball. The movementTick signal was created because the original clock speeds of 50MHz or 25MHz are far too fast for updating the dynamic elements' X and Y positions. At these speeds, updates would occur so rapidly that the human eye would not perceive smooth motion. The slower movementTick clock ensures that players can see the elements moving visibly on the screen.

The movementTick, SW3, SW2, and SW1 signals were used to update the positions of the dynamic elements. The horizontal and vertical pixel positions (hpos and vpos) were derived from the 25MHz clock, while the SW0 signal acted as a reset switch. During this process, the horizontal and vertical synchronization signals (H and V) were also calculated. Additionally, a bright signal was generated to indicate whether the currently tracked pixel position was within the active 640x480 video display area. The SW2 signal acts as a pause switch, in which all dynamic elements become static.

The derived signals were then used to determine the color of the currently tracked pixel. These 8-bit red, green, and blue channels were output to the Spartan-3E board for display.

Results

Timing Diagrams

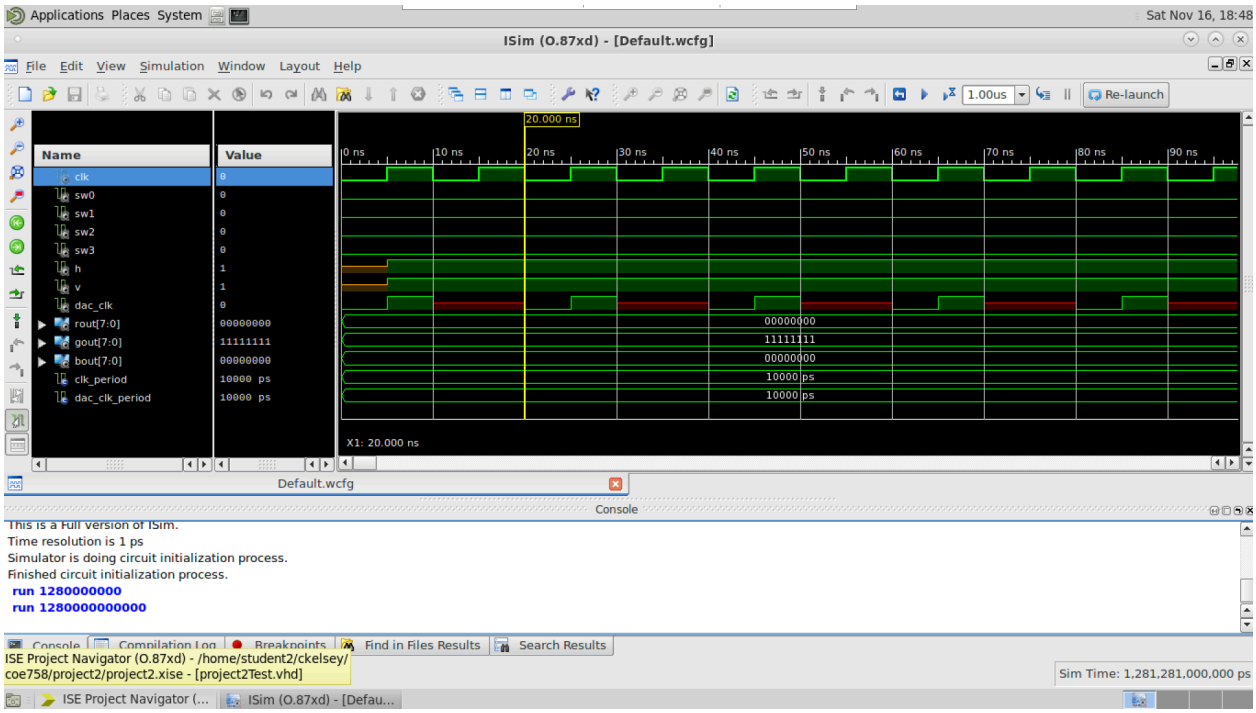

Figure 4: Timing diagram showcasing 50Mhz on board clock signal “clk” and derived 25MHz clock signal “DAC_CLOCK”

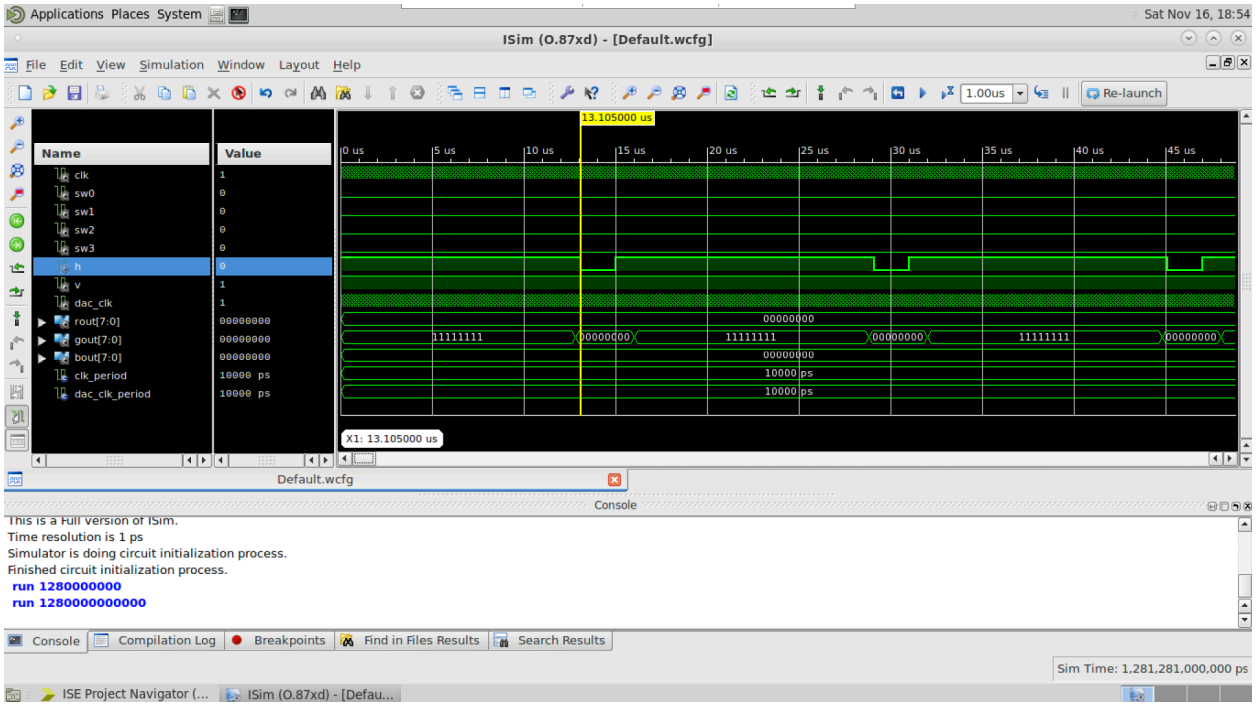

Figure 5: First HSync “H” active low signal occurs at time 13,105 ns, period between HSync signals is 16us

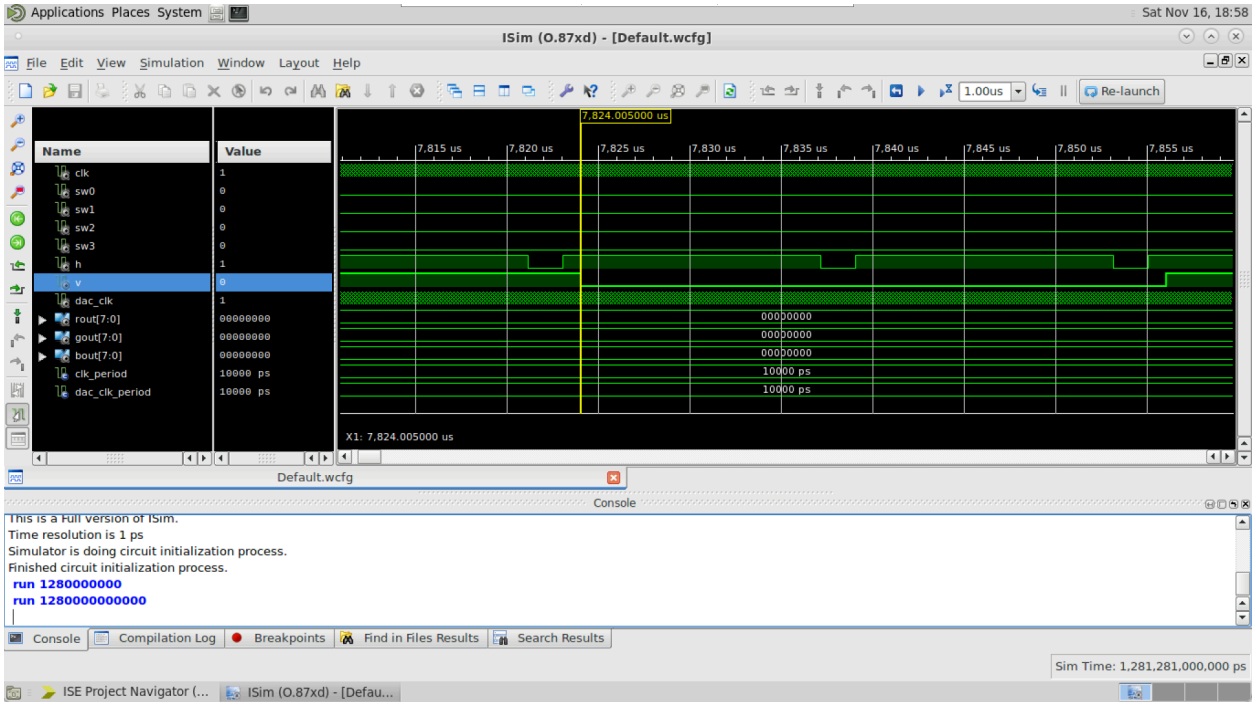

Figure 6: First VSync “V” active low signal occurs at time 7824us. 7824/16 = 489 ~=480. 480 is the number of horizontal retraces or HSync signals that need to occur before vertical retrace or a VSync signal occurs.

Screen Capture

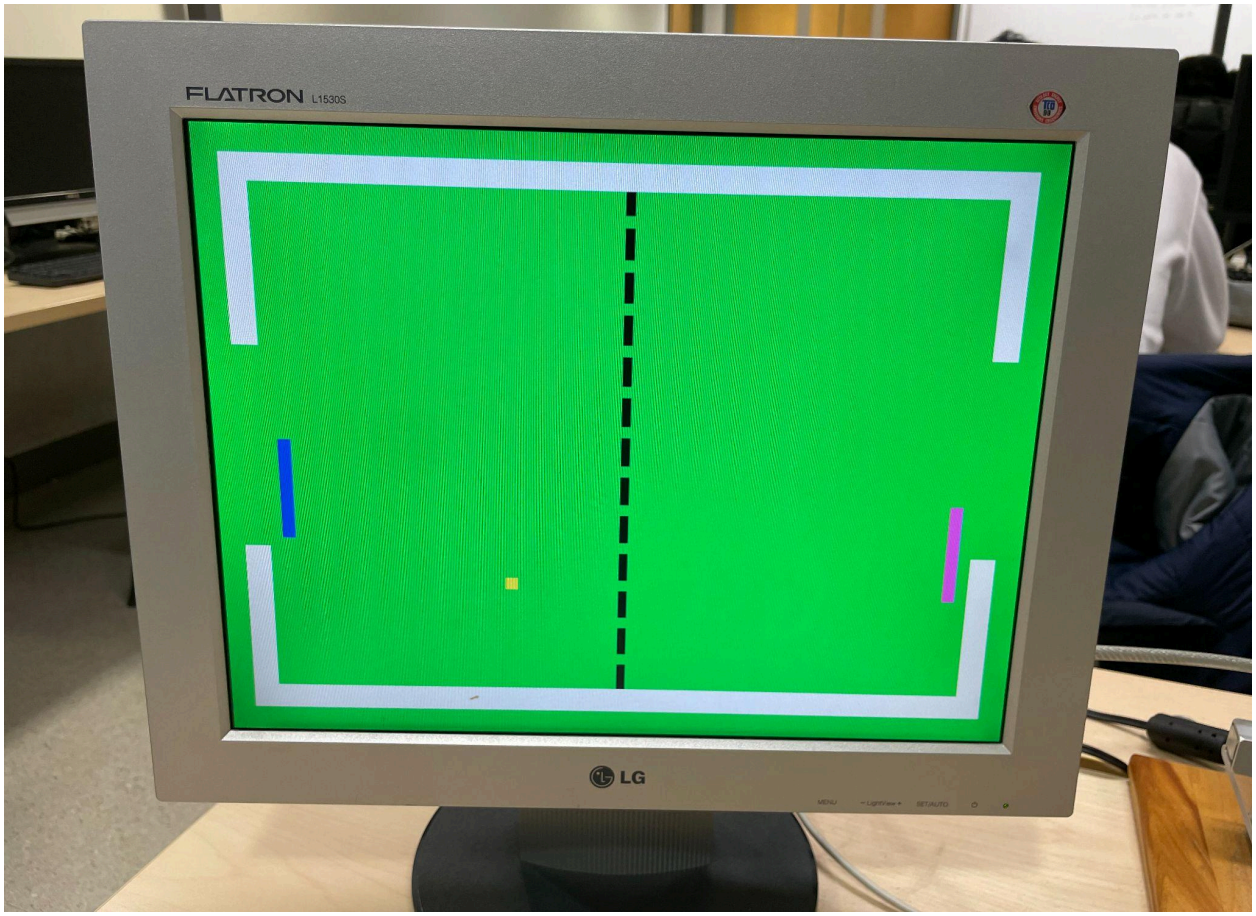

Figure 7: Image of the Pong game displayed on a VGA monitor

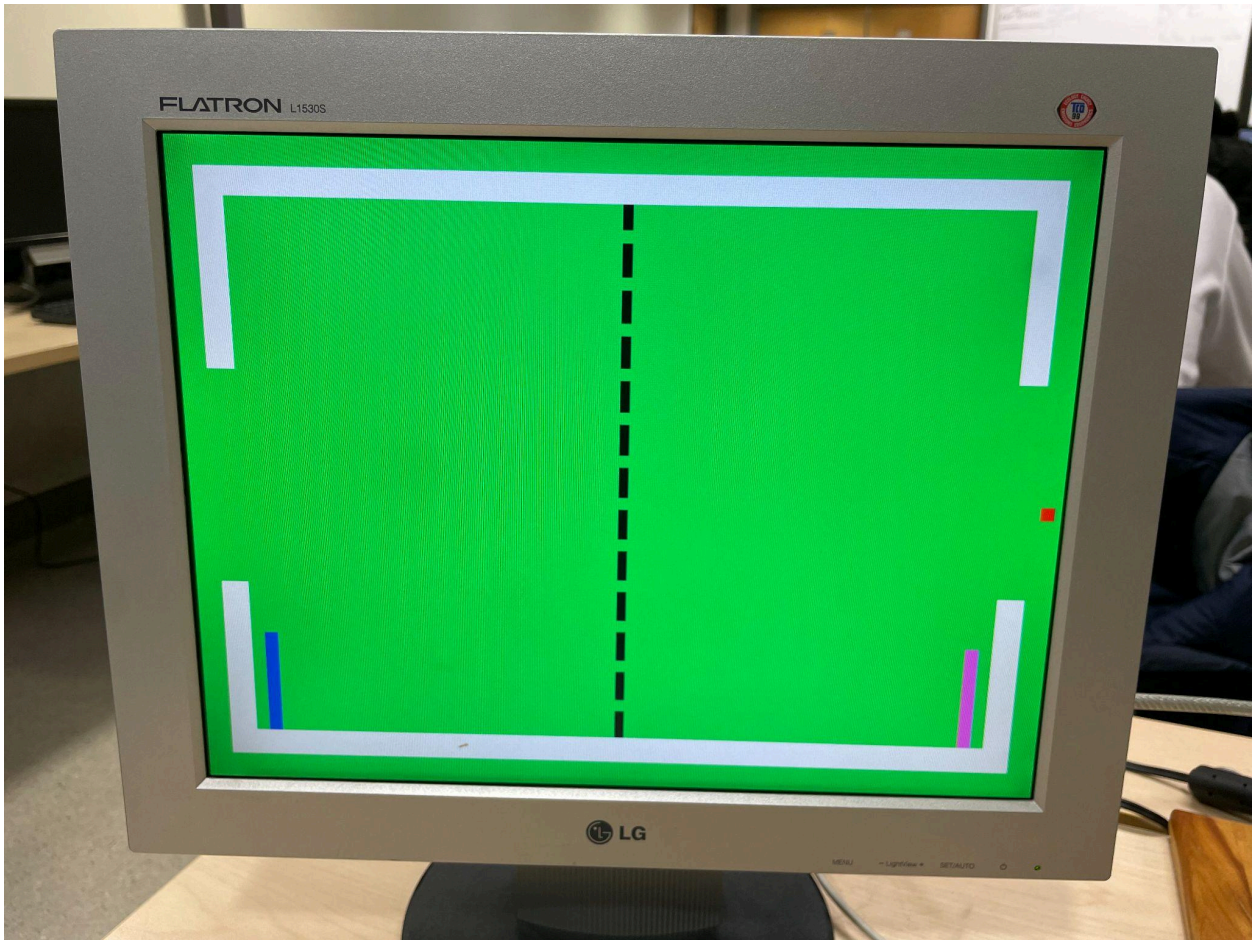

Figure 8: Image of the ball when it passes the “gate”

Conclusions

The simple video game for the VGA was successfully implemented. The video game functions as intended and responds accordingly to player controls. This project offered valuable insights into the fundamentals of the VGA standard, as well as video imaging and graphical interfaces.

References

[1] “COE758 -Digital Systems Engineering Project #2 -Simple Video Game Processor for VGA Objectives.” Accessed: Nov. 15, 2024. [Online]. Available: https://www.ecb.torontomu.ca/~lkirisch/ele758/labs/SimpleVideoGame%5b11-11-11%5d.pdf

[2] COE 758 Xilinx ISE 13.4, "Tutorial Functional Simulation," Tutorial, pp. 1–10, Nov. 2024.

[3] Karim Soubra, my TA who gave guidance throughout the labs.

[4] Anthony Andres, my friend who helped me with the pixel clocks, and ball collision.

Appendix

```
20  library IEEE;
21  use IEEE.STD_LOGIC_1164.ALL;
22  use IEEE.STD_LOGIC_ARITH.ALL;
23  use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25  entity project2 is
26      Port (  clk : in  STD_LOGIC;
27              SW0 : in  STD_LOGIC;
28              SW1 : in  STD_LOGIC;
29              SW2 : in  STD_LOGIC;
30              SW3 : in  STD_LOGIC;
31              H : out  STD_LOGIC;  --when low, retrace back to left
32              V : out  STD_LOGIC;  --when low, retrace back to top
33              DAC_CLK : out  STD_LOGIC;
34              Rout : out  STD_LOGIC_VECTOR (7 downto 0);
35              Gout : out  STD_LOGIC_VECTOR (7 downto 0);
36              Bout : out  STD_LOGIC_VECTOR (7 downto 0));
37  end project2;
38
39  architecture Behavioral of project2 is
```

```
40
41  -- VGA horizontal parameters
42      CONSTANT hFP : integer := 16;   --front porch
43      CONSTANT hBP : integer := 48;   --back porch
44      CONSTANT hEND : integer := 800; --complete line
45      CONSTANT hSP : integer := 96;   --sync pulse
46
47  -- VGA vertical parameters
48      CONSTANT vFP : integer := 10;
49      CONSTANT vBP : integer := 33;
50      CONSTANT vEND : integer := 525;
51      CONSTANT vSP : integer := 2;
52
53  -- clk signals
54      signal clk25 : STD_LOGIC := '0'; -- normal clk runs at 50Mhz, VGA 640x480 60fps(?) require
55      signal clk_counter : integer := 0;
56      signal movementTick : STD_LOGIC := '0';
57
58
59      signal bright : STD_LOGIC;
```

```
60      signal hpos : integer := 0; -- hpos and vpos keep track of current pixel position on scree
61      signal vpos : integer := 0; -- hpos counts from 0 to 639(horizontal), vpos from 0 to 479(v
62      signal leftBarX : integer := 50; -- left bar x position, default position at pixel 50 (alw
63      signal leftBarY : integer := 360;  -- represents bottom of bar
64      signal RightBarX : integer := 580; -- right bar x position, default position at pixel 580
65      signal RightBarY : integer := 60;
66      signal BallX : integer := 310;  -- represents left side of ball (default start on the righ
67      signal BallY : integer := 230;  -- represents the bottom of the ball
68      signal hMove : STD_LOGIC; -- determine if left or right movement (0 right, 1 left)
69      signal vMove : STD_LOGIC; -- determine if up or down movement (0 down, 1 up)
70
71  begin
72
73  --determine clk signals
74     process (clk)
75     begin
76        if (clk'Event and clk = '1') then
77           clk25 <= NOT clk25; --whenever 50Mhz clk has rising edge, invert the signal of clk25.
78           --clk 25 starts at 0, then rising edge clk, clk25=1, then next period of clk, clk25=0
79
```

```
80          if (clk_counter  = 180000) then
81             movementTick <= NOT movementTick;--same premise as clk>clk25 conversion, except mu
82             clk_counter <= 0;                 --50MHz clock is 50 million cycles per second, w
83          else
84             clk_counter <= clk_counter + 1;
85          end if;
86       end if;
87    end process;
88
89 --pixel tracking
90    process (clk25, SW0)
91    begin
92       -- Reset Switch
93       if (SW0 = '1') then
94          hpos <= 0;
95          vpos <= 0;
96          H <= '1';
97          V <= '0';
98          bright <= '0';
99       else
```

```
100         if (clk25'Event and clk25 = '1') then
101
102            --hpos vpos counters
103            if (hpos < hEND - 1) then --once hpos reaches 799 (ie it has gone through all 800
104               hpos <= hpos + 1;
105            else
106               hpos <= 0;
107            if (vpos < vEND - 1) then
108               vpos <= vpos + 1;
109            else
110                vpos <= 0;
111            end if;
112            end if;
113
114            -- Horizontal Sync
115            --if the hpos is within before the back porch
116            if ((hpos < 639 + hFP) OR (hpos >= 639 + hFP + hSP)) then -- |    Active display a
117               H <= '1';                                              -- |                H =
118            else
119               H <= '0'; --otherwise retrace occurs
```

```
120            end if;
121            -- if the vpos is within before the front porch
122            if ((vpos < 479 + vFP) OR (vpos >= 479 + vFP + vSP)) then
123               V <= '1';
124            else
125               V <= '0';--retrace occurs
126            end if;
127
128            --check if in display area, bright
129            if ((hpos < 640) AND (vpos < 480)) then
130               bright <= '1';                          -- |    Active display area  |  front p
131            else                                       -- |       bright = '1'       |
132               bright <= '0';
133            end if;
134         end if;
135      end if;
136   end process;
137
138   --MOVEMENT & COLLISION
139   process (movementTick, SW3, SW1, SW2, leftBarY, RightBarY)
```

```
140   begin
141      if (movementTick'Event and movementTick = '1') then
142
143         if (SW2 = '0') then --SW2 is low, game is not paused, movement is allowed
144
145            --left bar movement
146            if (SW1 = '0') then --when sw1 is up...
147               if (leftBarY < 360) then--...and leftbar is below 360(the bounds of the white b
148                  leftBarY <= leftBarY + 1;--...move the bar up one pixel
149               else
150                  leftBarY <= 360;--otherwise if it is at the border, keep the bar at that pos
151               end if;
152            else
153               if (leftBarY > 40) then-- when sw1 is down and leftbar is above the bottom whit
154                  leftBarY <= leftBarY - 1;--...move the bar down 1 pixel
155               else
156                  leftBarY <= 40;
157               end if;
158            end if;
159
160            --right bar movement
161            if (SW3 = '0') then -- same logic as left bar movement
162               if (RightBarY < 360) then
163                  RightBarY <= RightBarY + 1;--move 1 pixel up
164               else
165                  RightBarY <= 360;
166               end if;
167            else
168               if (RightBarY > 40) then
169                  RightBarY <= RightBarY - 1;--move 1 pixel down
170               else
171                  RightBarY <= 40;
172               end if;
173            end if;
174
175            -- COLLISION
176            if ( BallX >= leftBarX and BallX < leftBarX + 10) then
177               -- when ball hits left player, ball is moving left (1), change direction to rig
178               if (((BallY >= leftBarY) or (BallY + 10 >= leftBarY)) and ((BallY < leftBarY +
179                  hMove <= '0';
180               end if;
181            elsif ( BallX = 40 ) then
182               -- Change direction when hit left boundary
183               if ( (BallY >= 40 and BallY < 160) or (BallY + 10 >= 360 and BallY + 10 < 440)
184                  hMove <= '0';
185               end if;
186            elsif (BallX + 10 > RightBarX and BallX + 10 <= RightBarX + 10) then
187               -- Change direction when hit the right player
188               if ( ((BallY >= RightBarY) or (BallY + 10 >= RightBarY)) and ((BallY < RightBar
189                  hMove <= '1';
190               end if;
191            elsif (BallX + 10 = 600) then
192               -- Change direction when hit right boundary
193               if ( (BallY >= 40 and BallY < 160) or (BallY + 10 >= 360 and BallY + 10 < 440)
194                  hMove <= '1';
195               end if;
196            end if;
197            if ( BallY - 1 <= 40 ) then --the ball only changes vertical direction when hittin
198               vMove <= '1';              --the top boundary or the bottom boundary at 40 and 4
199            elsif (BallY + 11 >= 440) then
```

```
            vMove <= '0';
         end if;

         -- Update ball location
         -- BallX is the right side of the ball, the ball is 10 pixels wide
         if ((BallX > 0) and (BallX + 10) < 639) then --when the ball is within the display
            -- Horizontal
            if (hMove = '0') then--if hmov indicates right movement
               BallX <= BallX + 1;--ball keeps going to the right
            elsif (hMove = '1') then--if hmov indicates left movement
               BallX <= BallX - 1;--ball keeps going to the left
            end if;
            -- Vertical
            if (vMove = '0') then--same logic as above
               BallY <= BallY - 1;
            elsif(vMove = '1') then
               BallY <= BallY + 1;
            end if;
         else
            BallX <= 300; --set default posiiton for ball if something happens
            BallY <= 220;
         end if;
      end if;
   end if;
end process;

--set colours when pixel is within the display area
process (bright)
begin
   if (bright = '0') then
      Rout <= (others => '0');
      Gout <= (others => '0');
      Bout <= (others => '0');
   else
      if (hpos >= 20 and hpos < 640 - 20 and vpos >=20 and vpos < 480 - 20) then
         if ( vpos < 40 or vpos >= 440) then --top 20..40 bottom 440..460
            Rout <= (others => '1');  -- Display white for top & bottom border
            Gout <= (others => '1');
            Bout <= (others => '1');
         elsif ((hpos < 40 OR hpos >= 640 - 40) and (vpos < 160 or vpos >= 320)) then --top
            Rout <= (others => '1');  -- Display white for left & right border
            Gout <= (others => '1');
            Bout <= (others => '1');
         elsif ((hpos >= BallX and hpos < BallX + 10) and (vpos >= BallY and vpos < BallY +
            Rout <= (others => '1');   -- Color Ball inside play field (gate + border)
            Gout <= (others => '1');
            Bout <= (others => '0');
         elsif ((hpos >= leftBarX and hpos < leftBarX + 10) and (vpos >= leftBarY and vpos
            Rout <= (others => '0');   -- Color Player 1
            Gout <= (others => '0');
            Bout <= (others => '1');
         elsif ((hpos >= RightBarX and hpos < RightBarX + 10) and (vpos >= RightBarY and vp
            Rout <= (others => '1');   -- Color Player 2
            Gout <= (others => '0');
            Bout <= (others => '1');
         elsif (vpos >= 40 and vpos < 440 and hpos > 316 and hpos <324) then --height 40..4
            --if(((vpos+7) mod 16) = 0 or ((vpos+6) mod 16) = 0 or ((vpos+5) mod 16) = 0 or
            if(((vpos+7) mod 32) = 0 or ((vpos+6) mod 32) = 0 or ((vpos+5) mod 32) = 0 or (
               Rout <= (others => '0');   -- within an 8 pixel line in the middle between t
               Gout <= (others => '1');   -- if a pixel position
```

```
260                  Bout <= (others => '0');
261               else
262                  Rout <= (others => '0');   -- black center line
263                  Gout <= (others => '0');
264                  Bout <= (others => '0');
265               end if;
266            else
267               Rout <= (others => '0');    -- Color background
268               Gout <= (others => '1');
269               Bout <= (others => '0');
270            end if;
271         elsif ((hpos >= BallX and hpos < BallX + 10) and (vpos >= BallY and vpos < BallY + 10
272            Rout <= (others => '1');   -- Color of ball when it goes out
273            Gout <= (others => '0');
274            Bout <= (others => '0');
275         else
276            Rout <= (others => '0');    -- Color background
277            Gout <= (others => '1');
278            Bout <= (others => '0');
279         end if;
280      end if;
281   end process;
282
283   DAC_CLK <= clk25;
284
285end Behavioral;
```